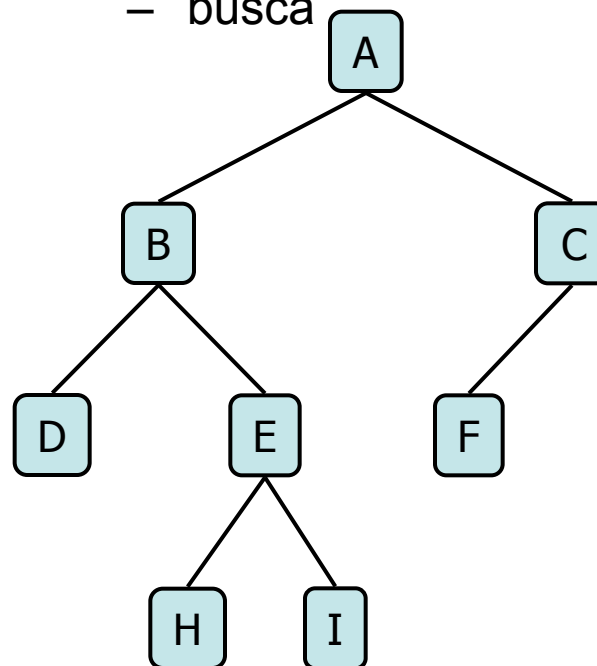


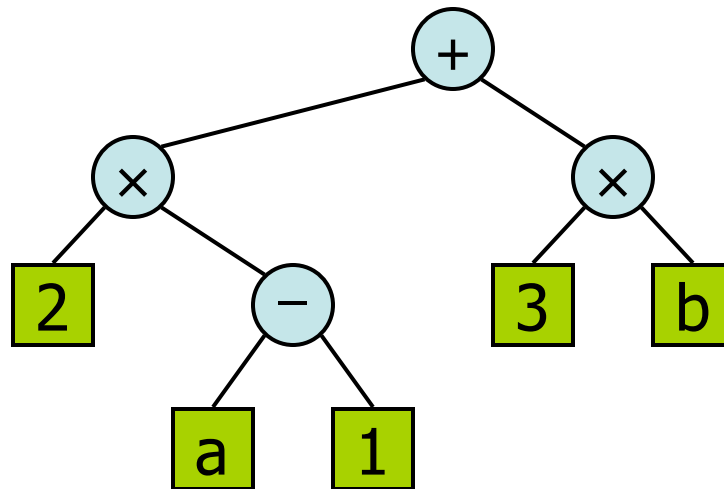
Árvore binária - revisão

- Uma árvore binária é uma árvore com as seguintes propriedades:
 - Cada nó interno tem, no máximo, dois filhos
 - Árvore binária própria é aquela em que cada nó tem exatamente zero ou dois filhos
 - Os filhos de um nó são um par ordenado
- Chamamos os filhos de um nó de filho da esquerda e filho da direita
- Podemos, também, definir uma árvore binária recursivamente como:
 - uma árvore consistindo de um único nó, ou
 - Uma árvore cuja raiz tem um par ordenado de filhos, cada um dos quais é uma árvore binária
- Aplicações:
 - Expressões aritméticas
 - Processo de decisão
 - busca



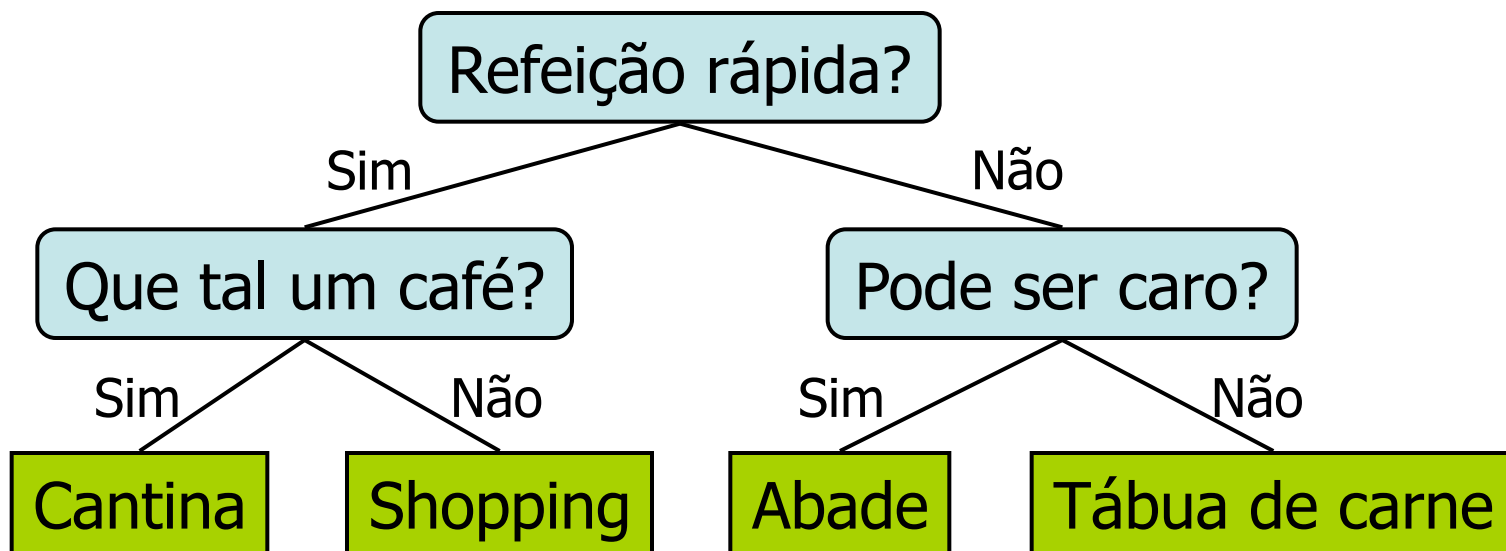
Árvore de expressões aritméticas

- Árvore binária associada com uma expressão aritmética
 - Nós internos: operadores
 - Nós externos: operandos
- Exemplo: árvore da expressão aritmética para a expressão $(2 \times (a - 1) + (3 \times b))$



Árvore de decisão

- Árvore binária associada com um processo de decisão
 - Nós internos: questões com respostas sim/não
 - Nós externos: decisões
- Exemplo: Onde jantar



Propriedades de AB (BT)

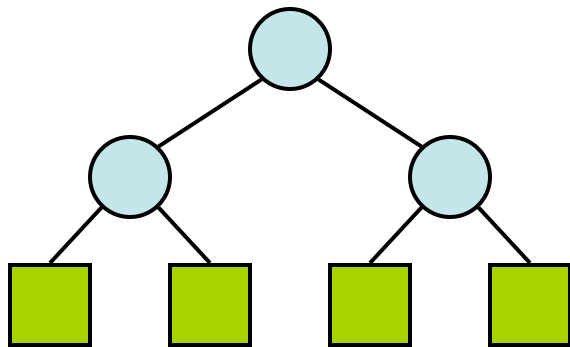
- Notação

n número de nós

e número de nós
externos

i número de nós
internos

h altura (*height*)



- Propriedades:

- $e = i + 1$

- $n = 2e - 1$

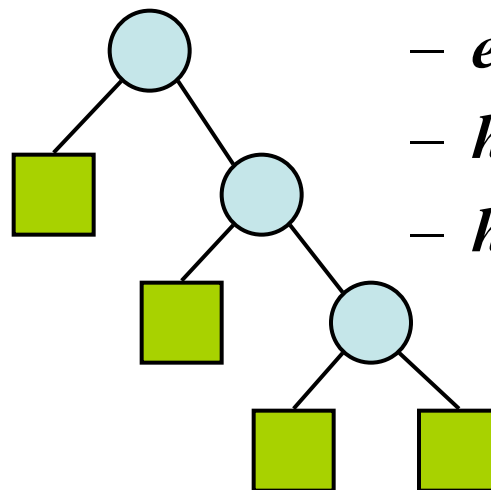
- $h \leq i$

- $h \leq (n - 1)/2$

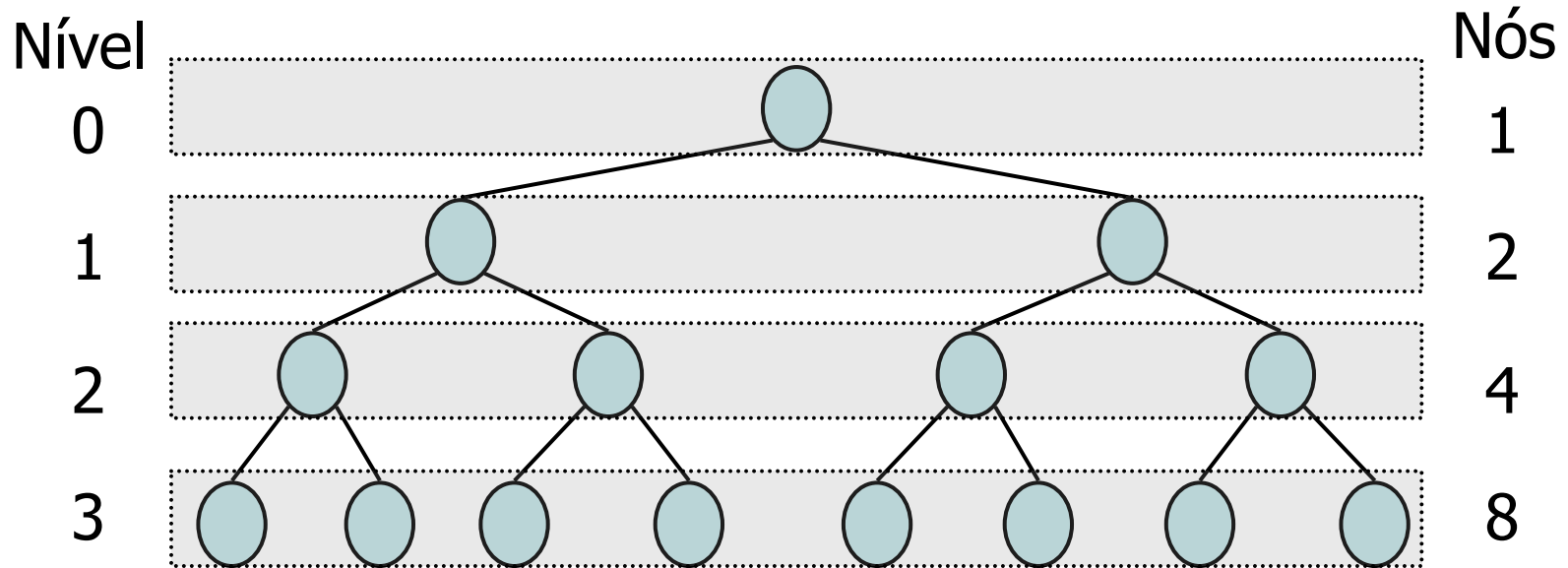
- $e \leq 2^h$

- $h \geq \log_2 e$

- $h \geq \log_2 (n + 1) - 1$



Propriedades de AB (BT)



Número máximo de nós em um nível h é 2^h

Número total de nós é, no máximo é $2^{h+1} - 1$

TAD Árvore binária

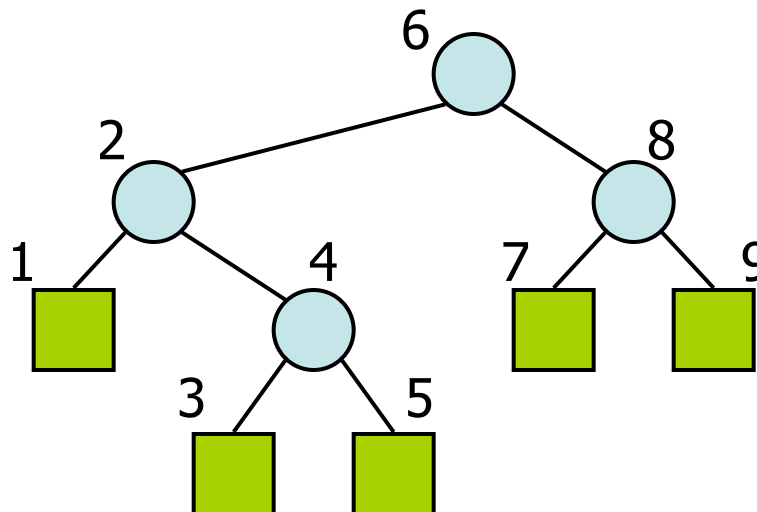
- O TAD ArvoreBinaria possui os métodos de árvore.
- Métodos adicionais:
 - No getFilhoEsquerdo()
 - No getFilhoDireito()
 - boolean éFilhoEsquerdo()
 - boolean éFilhoDireito()
- Métodos de atualização podem ser definidos por estruturas de dados que implementam o TAD ArvoreBinaria

Travessia em ordem

- Na travessia inorder, um nó é visitado depois do filho da esquerda e antes do filho da direita
- Aplicação: Desenhar uma árvore binária
 - $x(v)$ = rank de v
 - $y(v)$ = profundidade de v

Algoritmo *inOrder*(v)

```
se ( $v.getFilhoEsquerdo() \neq null$ )  
    inOrder( $v.getfilhoEsquerdo()$ );  
visite( $v$ );  
se ( $v.getFilhoDireito() \neq null$ )  
    inOrder( $v.getfilhoDireito()$ );
```



Travessia pré ordem

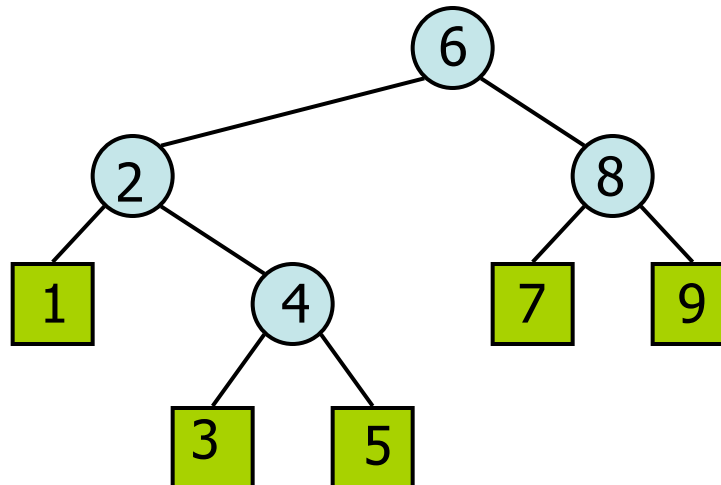
- Na travessia pré-order, um nó é visitado antes do filho da esquerda e antes do filho da direita

Algoritmo *preOrder(v)*

```
visite(v);
```

```
se (v.getFilhoEsquerdo() != null)  
    inOrder(v.getfilhoEsquerdo());
```

```
se (v.getFilhoDireito() != null)  
    inOrder(v.getfilhoDireito());
```

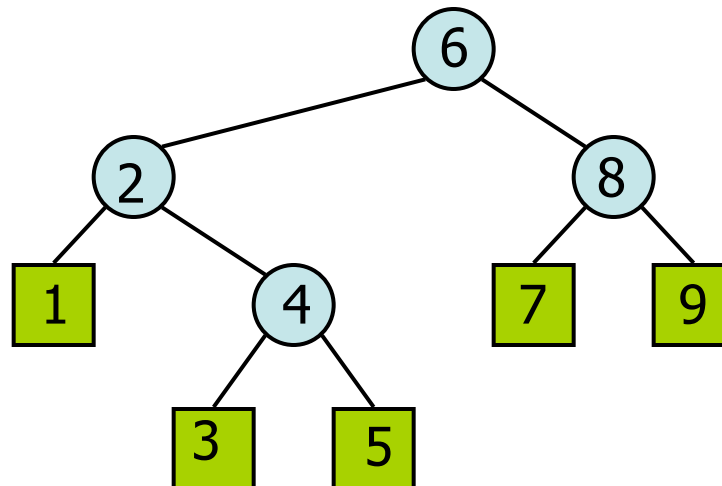


Travessia pós ordem

- Na travessia pós-order, um nó é visitado depois do filho da esquerda e depois do filho da direita

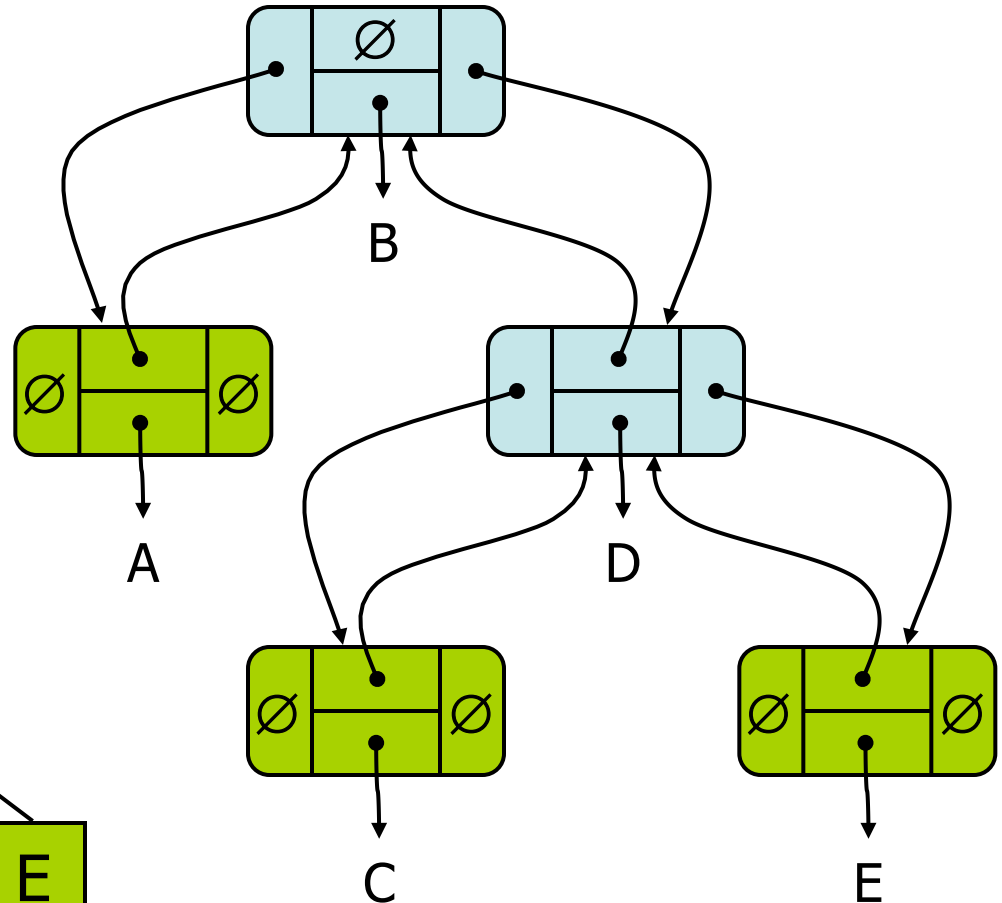
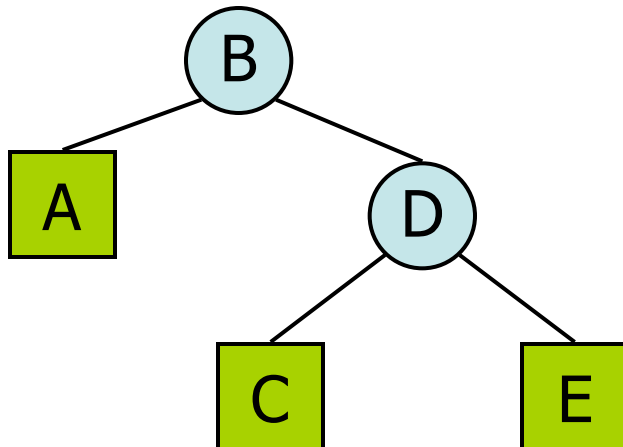
Algoritmo *posOrder(v)*

```
se (v.getFilhoEsquerdo() != null)  
    inOrder(v.getfilhoEsquerdo());  
se (v.getFilhoDireito() != null)  
    inOrder(v.getfilhoDireito());  
visite(v);
```



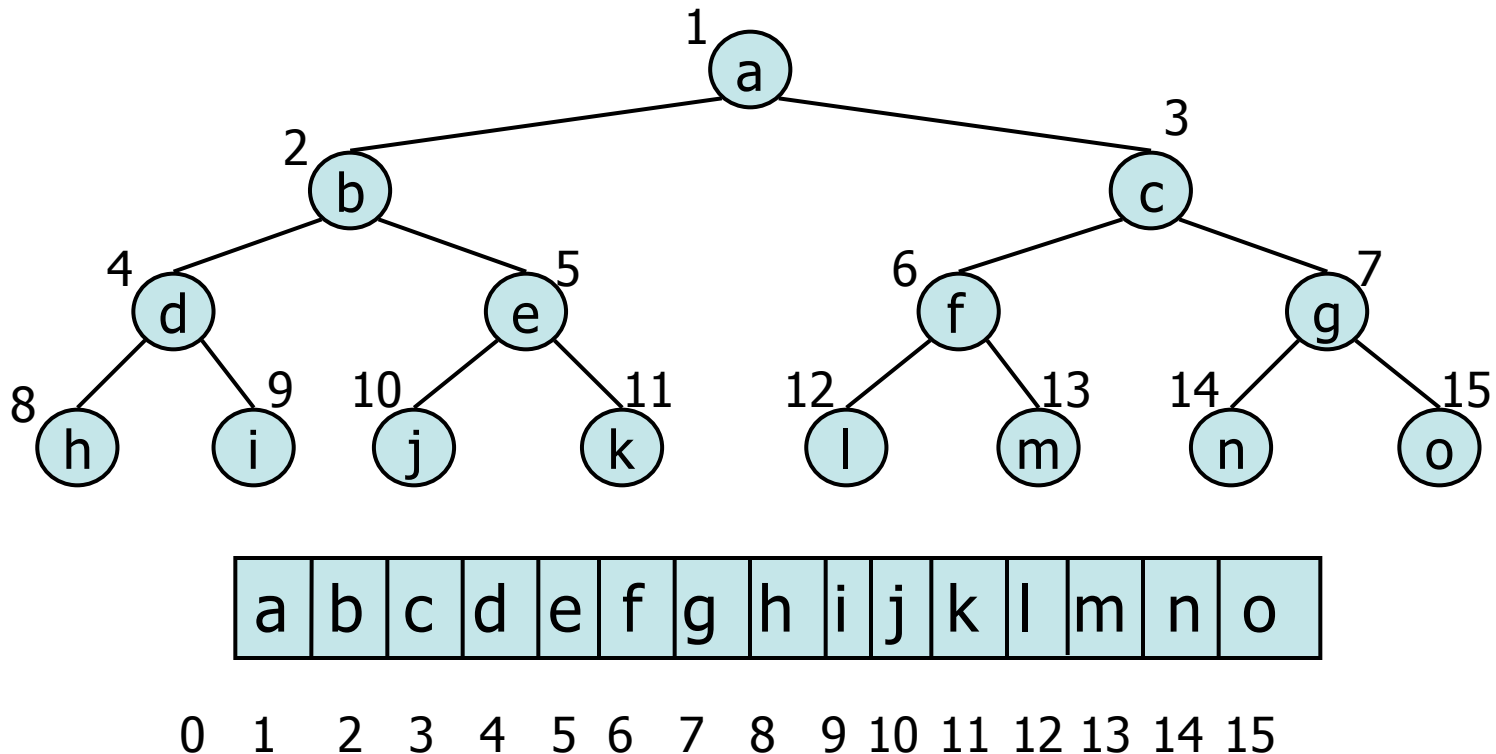
Estrutura de dados para AB

- Um nó é um objeto que armazena
 - Elemento
 - Nó pai
 - Filho da esquerda
 - Filho da direita
- Objetos nós implementam o TAD Position

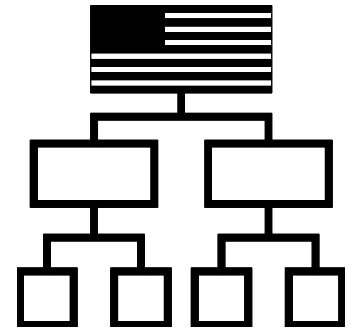


Estrutura de dados para AB

- Podemos usar um *array*



Árvore de pesquisa binária



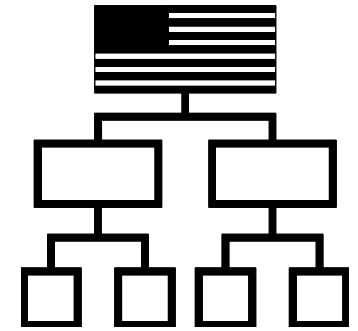
- Definição: seja $S=\{S_1, \dots, S_n\}$ o conjunto de chaves satisfazendo $S_1 < \dots < S_n$. Seja T uma árvore binária rotulada, com as seguintes características:
 - (i) T possui n nós. Cada nó v corresponde a uma chave distinta S_j pertencente a S e possui como rótulo o valor $r(v)=S_j$
 - (ii) Seja um nó v de T . Seja também v_1 , pertencente à subárvore esquerda de v . Então

$$r(v_1) < r(v)$$

analogamente, se v_2 pertence à subárvore direita de v ,

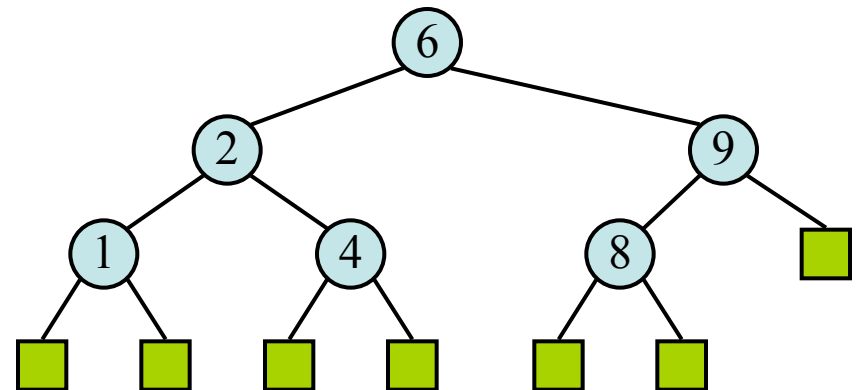
$$r(v_2) > r(v)$$

Árvore de pesquisa binária

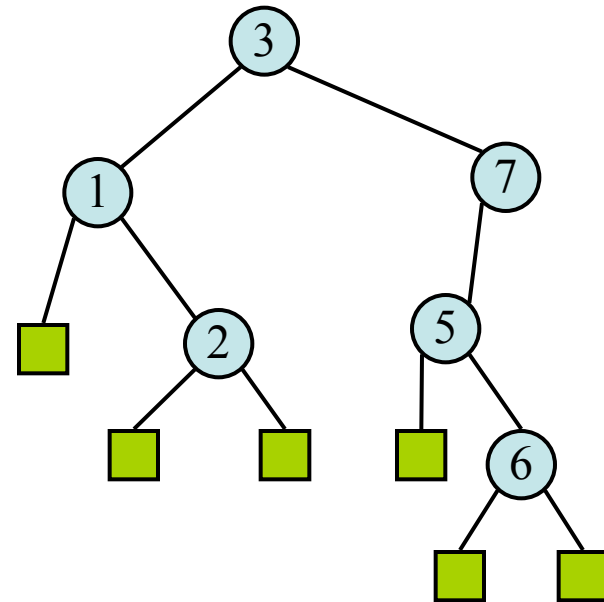
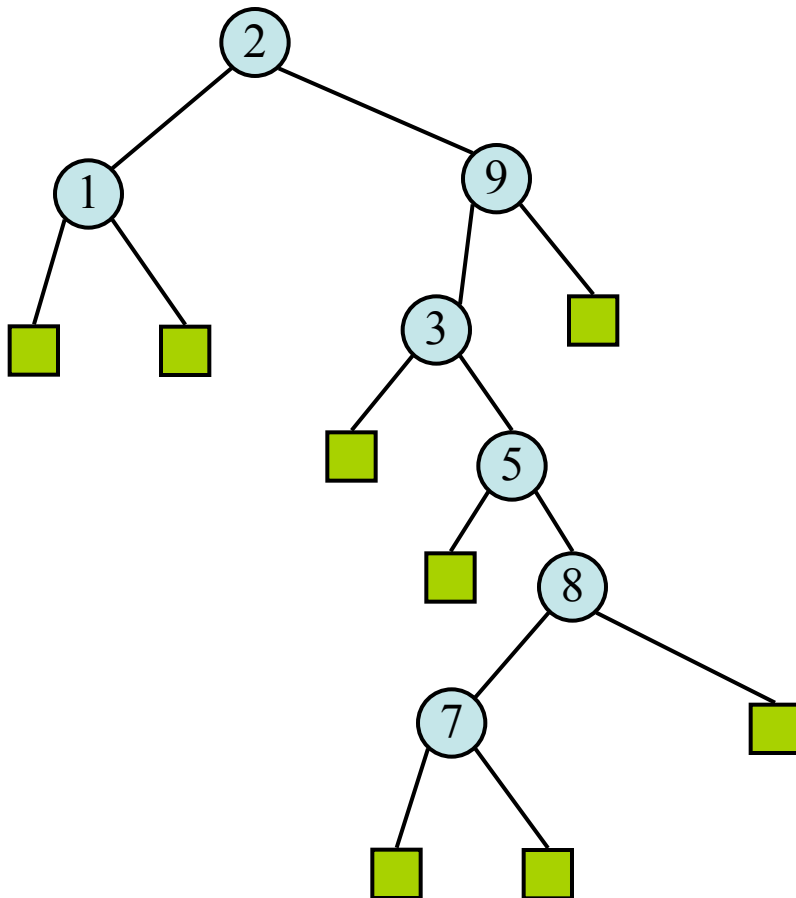


- Uma árvore de pesquisa binária é uma árvore binária armazenando chaves (ou itens) em seus nós internos e satisfazendo a seguinte propriedade:
 - Seja u , v e w três nós tais que u é nó esquerdo de v e w é o nó direito. Temos $key(u) \leq key(v) \leq key(w)$
- Nós externos não armazenam itens (null)

- Uma travessia em ordem visita as chaves em ordem crescente



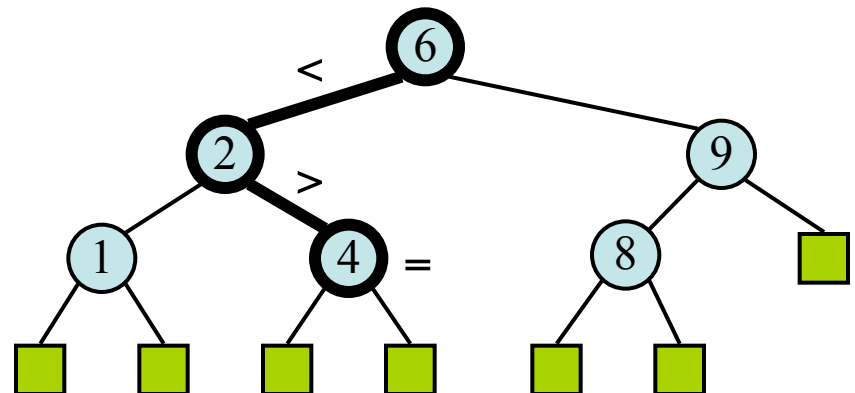
Árvore de pesquisa binária



Busca

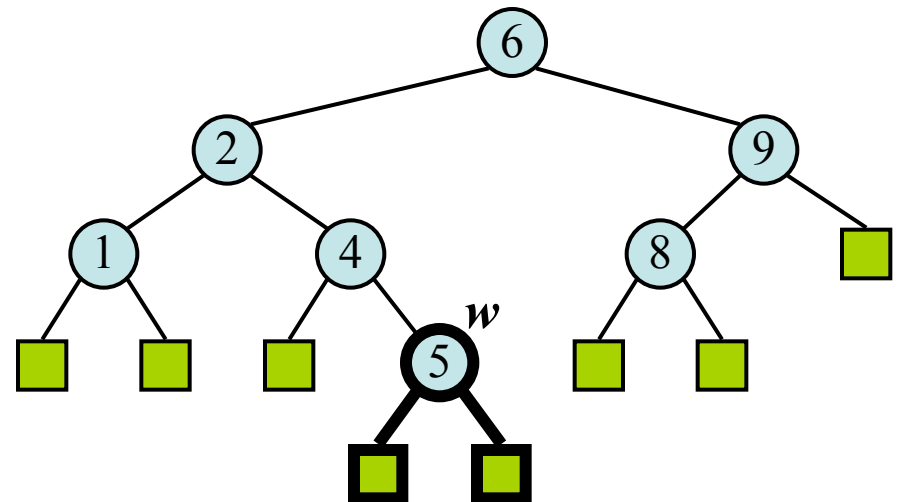
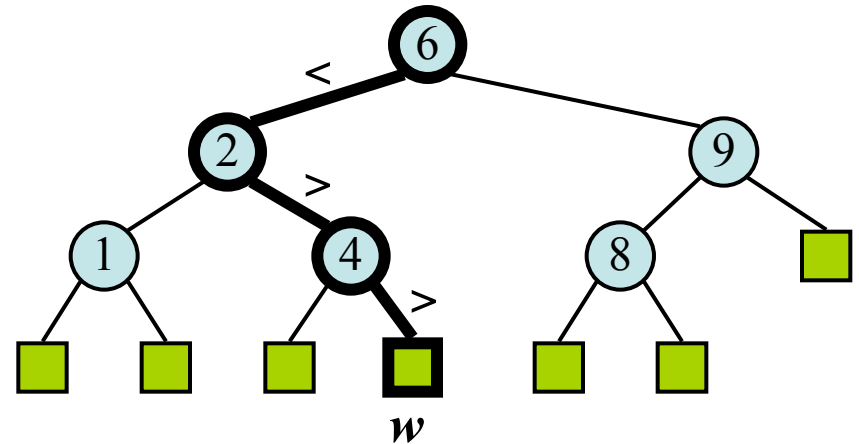
- Para procurar uma chave k , procuramos a partir da raiz comparando com a chave do nó.
- O próximo nó depende da comparação da chave pesquisada com a chave do nó atual
- Se chegar em uma folha e não encontrar a chave, retorna-se null
- Exemplo: find(4):
 - chama algoritmo `pesquise(getRaiz(),4)`

```
Algoritmo pesquise( $v, k$ )  
  se  $v.isExternal()$   
    retorne  $v$   
  se  $k < v.getChave()$   
    retorne  
  pesquise( $v.getFilhoEsquerdo(), k$ )  
  senão se  $k = v.getChave()$   
    retorne  $v$   
  senão {  $k > v.getChave()$  }  
    retorne  
  pesquise( $v.getFilhoDireito(), k$ )
```



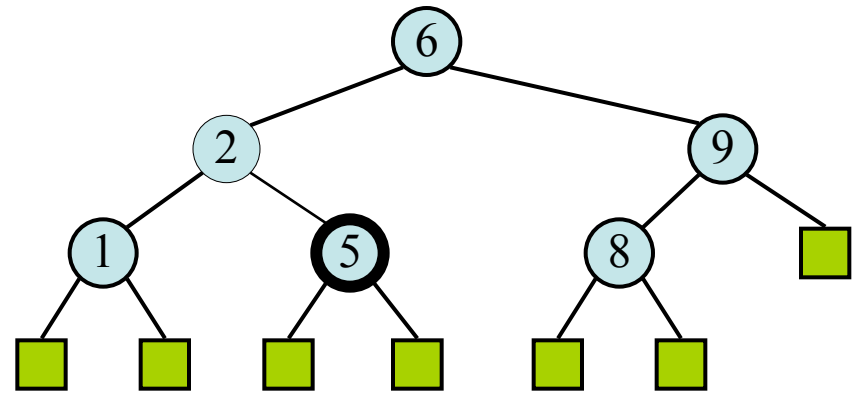
inserção

- Para executar $\text{inserção}(k, o)$, procura-se pela chave k
- Assumindo que k ainda não está na árvore, w será a folha encontrada pela busca
- Inserimos k no nó w
- Exemplo: inserir 5



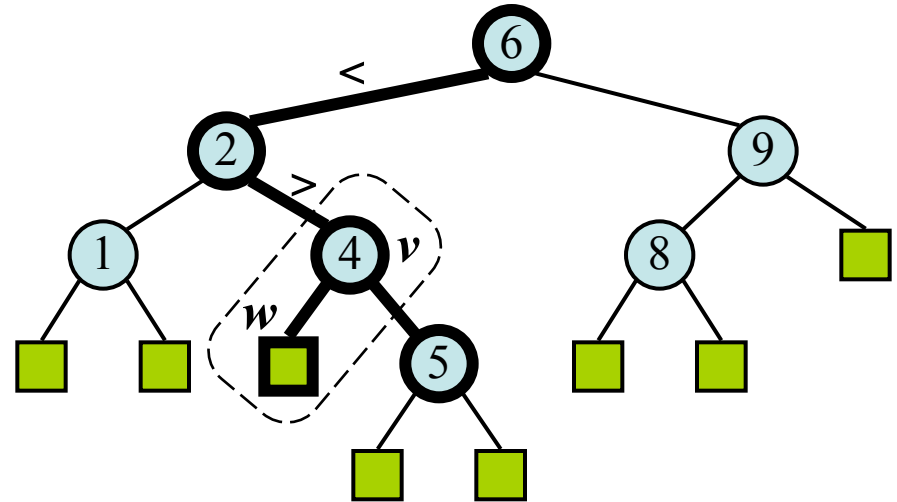
remoção

- Para executar $\text{remove}(k)$, procuramos pela chave k
- *Se o nó for folha remova-o.*
- *Remover(4)*

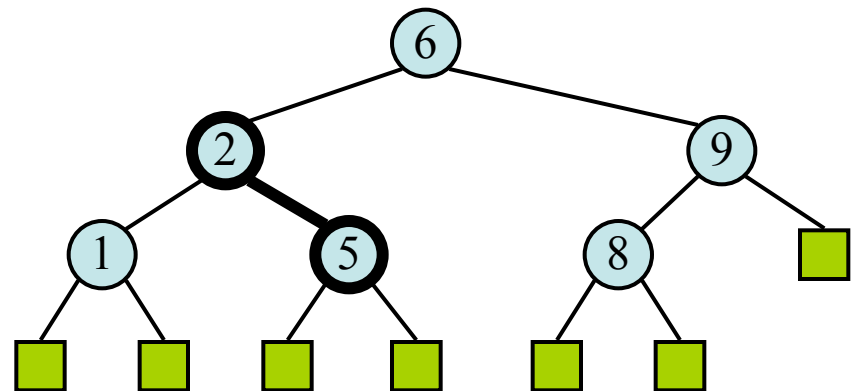


remoção

- *Se o nó tiver apenas um filho, remova-o e o seu filho assume o seu lugar*

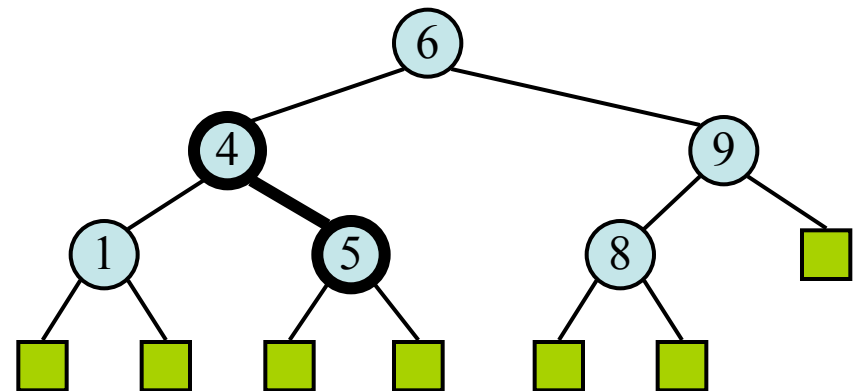
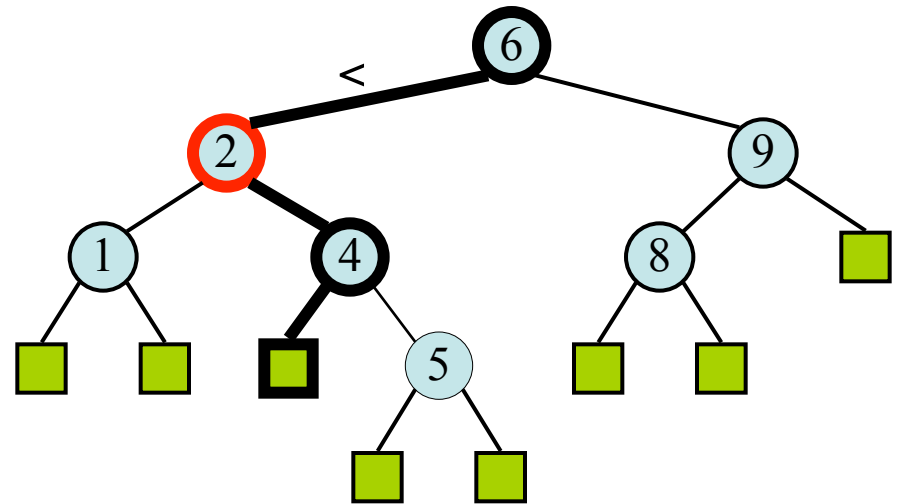


- *Remove(4)*



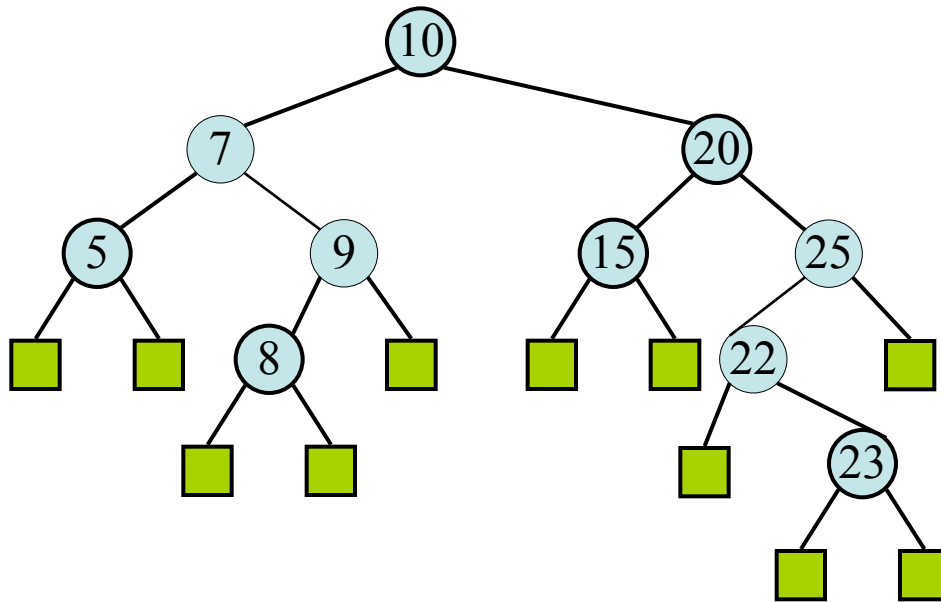
remoção

- *Se o nó tiver dois filhos, encontre o sucessor através de um caminhamento em-ordem na subárvore direita do nó a ser removido.*
- *O nó sucessor assume o lugar do nó removido*
- *Remover(2)*



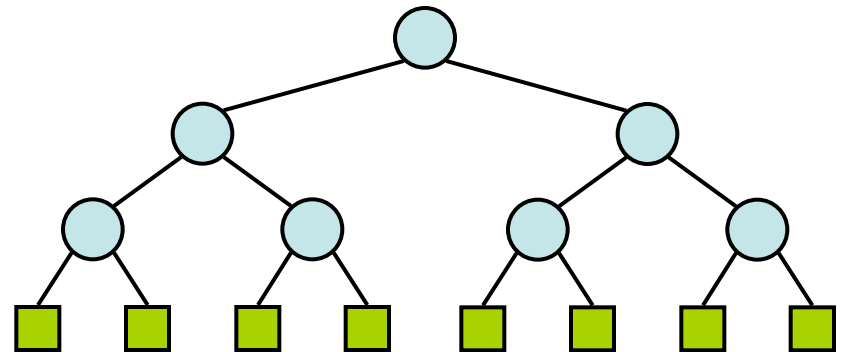
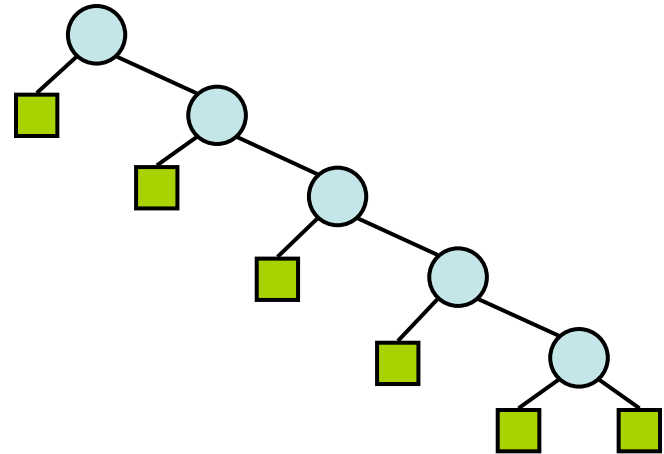
Exemplo de remoção

- *Remover 5, 9 e 20*



Desempenho

- Considere um dicionário com n itens, implementado com uma árvore binária de pesquisa de altura h
 - o espaço usado é $O(n)$
 - métodos procura, insere e remove executam em tempo $O(h)$
- A altura h é $O(n)$ no pior caso e $O(\log n)$ no melhor caso



Desempenho

- Uma árvore binária de pesquisa que tem nós sendo inseridos e apagados de forma aleatória tem em média um custo de operação de $O(\log n)$

